6GGW / NetSwitch — database connections, tables & connectors

The data plane is Microsoft SQL Server (2016–2025; the T-SQL is standard enough to port to Oracle / PostgreSQL with only dialect changes). This document is the table map, the relationships between them, and a ready-to-paste connector for every stack you are likely to use. All three tables and the scaling functions are created by `netswitch-sql/scaling/ggw_scaling.sql`.

Table map (what connects to what)

```
device_session                sccm_block  1 ──< N  scaler_result
(one row per device join)      (one 8x8 measure)    (one row per scaler tried)
seq (PK)                       block_id (PK) ──────  block_id (FK → sccm_block)
user, code_idx (burned code)   captured_at           scaler_q  (PK part)
ip gw mask dns dns2            coeffs (64 numbers)    rate_bits, psnr_db
dns_periph dhcp link           powered / unpowered    chosen (BIT: best pick)
```

- `device_session` is standalone — the registrar writes one row each time a device enrols with a burned backup code (IP / GW / MASK / DNS / DNS2 / DNS-peripheral / DHCP / link=ota|cable).
- `sccm_block` holds one captured 8×8 measure (the 64 DCT coefficients from `ggw_ddtm`) and its powered/unpowered split.
- `scaler_result` has a **foreign key** `block_id → sccm_block.block_id`; one row per scaler Q tried, with the single winner flagged `chosen = 1`.

Table DDL (exactly as shipped)

```
CREATE TABLE dbo.device_session (
    seq         INT IDENTITY(1,1) PRIMARY KEY,
    logged_at   DATETIME2     NOT NULL DEFAULT SYSUTCDATETIME(),
    [user]      NVARCHAR(64)  NOT NULL,
    code_idx    NVARCHAR(16)  NOT NULL,      -- which backup code (#N), already burned
    ip NVARCHAR(45), gw NVARCHAR(45), mask NVARCHAR(45),
    dns NVARCHAR(45), dns2 NVARCHAR(45), dns_periph NVARCHAR(45),
    dhcp NVARCHAR(16), link NVARCHAR(16)    -- 'ota' | 'cable'
);

CREATE TABLE dbo.sccm_block (
    block_id    INT IDENTITY(1,1) PRIMARY KEY,
    captured_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    note        NVARCHAR(128),
    coeffs      NVARCHAR(MAX) NOT NULL,      -- 64 space-separated DCT coefficients
    powered INT, unpowered INT
);

CREATE TABLE dbo.scaler_result (
    block_id  INT   NOT NULL REFERENCES dbo.sccm_block(block_id),
    scaler_q  FLOAT NOT NULL
    rate_bits FLOAT NOT NULL
    psnr_db   FLOAT NOT NULL
    chosen    BIT   NOT NULL DEFAULT 0,
    CONSTRAINT PK_sr PRIMARY KEY (block_id, scaler_q)
);
```

Connection strings (fill in host / db / credentials)

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.

Driver / stack	Connection string
ODBC (Driver 18)	`Driver={ODBC Driver 18 for SQL Server};Server=tcp:HOST,1433;Database=netswitch;Uid=USER;Pwd=***;Encrypt=yes;TrustServerCertificate=no;`
ADO.NET	`Server=tcp:HOST,1433;Database=netswitch;User ID=USER;Password=***;Encrypt=True;`
JDBC	`jdbc:sqlserver://HOST:1433;databaseName=netswitch;user=USER;password=***;encrypt=true;`
Azure SQL (hosted)	same as ADO.NET with `HOST=yourserver.database.windows.net`

Port **1433/tcp** is the default; open it in the firewall (see the install doc). Use `Encrypt=yes` in production and a real server certificate rather than `TrustServerCertificate`.

Connectors — paste-ready

C++ / ODBC (this is what `netswitch-sql/netswitch_sql.cpp` already uses):

```
SQLDriverConnect(hdbc, NULL,
  (SQLCHAR*)"Driver={ODBC Driver 18 for SQL Server};Server=tcp:HOST,1433;"
            "Database=netswitch;Uid=USER;Pwd=***;Encrypt=yes;",
  SQL_NTS, NULL, 0, NULL, SQL_DRIVER_NOPROMPT);
```

Python (pyodbc)

```
import pyodbc
cn = pyodbc.connect("Driver={ODBC Driver 18 for SQL Server};Server=tcp:HOST,1433;"
                    "Database=netswitch;Uid=USER;Pwd=***;Encrypt=yes;")
for row in cn.cursor().execute(
        "SELECT scaler_q, psnr_db FROM dbo.scaler_result WHERE chosen = 1"):
    print(row.scaler_q, row.psnr_db)
```

.NET (Microsoft.Data.SqlClient)

```
using var cn = new SqlConnection(
    "Server=tcp:HOST,1433;Database=netswitch;User ID=USER;Password=***;Encrypt=True;");
cn.Open();
using var cmd = new SqlCommand("EXEC dbo.register_session @user=@u, @ip=@ip", cn);
cmd.Parameters.AddWithValue("@u", "device-01");
cmd.Parameters.AddWithValue("@ip", "203.0.113.10");
cmd.ExecuteNonQuery();
```

PHP (sqlsrv)

```
$cn = sqlsrv_connect("HOST", [
    "Database"=>"netswitch","Uid"=>"USER","PWD"=>"***","Encrypt"=>true]);
$q = sqlsrv_query($cn, "SELECT TOP 10 * FROM dbo.device_session ORDER BY logged_at DESC");
```

Node.js (mssql / tedious)

```
const sql = require('mssql');
await sql.connect({ server:'HOST', database:'netswitch',
  user:'USER', password:'***', options:{ encrypt:true } });
const r = await sql.query`SELECT scaler_q, psnr_db FROM dbo.scaler_result WHERE chosen = 1`;
```

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.

Stored entry points (call these, don’t write raw INSERTs)

Proc / function	Use
`dbo.register_session @user,@code_idx,@ip,...`	insert one device_session row on enrol
`dbo.choose_best_scaler @block_id`	run the scaler sweep in-DB, flag the winner
`dbo.ddtm_constant()`	the precompute-once constant
`dbo.hyper_downscale / thrice_sinwave / lnlog`	the scaling maths (see MATH-INDEX)

Give me your real SQL host/instance name and I will hand you the exact tested connection string plus a 10-line smoke test that connects, creates the tables, and round-trips one row.

AI2ORBIT CO. Network system Pat. pend. Founded in 2018. Copyright 2018-2027. All rights reserved.